

Universidad Diego Portales

Facultad de Ingeniería y Ciencias

Curso Arquitectura de Software

Sistema de Gestión de Inventario Omnicanal (Sync-Stock)

Informe de 2da Entrega

Organización: Tienda de Pesca "PescaTotal"

Área: Departamento de Ventas e Inventario

Responsables: Fernando Llancao, Carlos Jara, Bastian Peña, Nicolas Canelo

Fecha: 28 de Marzo de 2026

Ubicación: Región Metropolitana, Chile

Semestre 1/2026

Índice

1. Equipo de Trabajo	2
2. Descripción del Sistema y Contexto Organizacional	2
2.1. La Organización: PescaTotal	2
2.2. Área de Trabajo: Departamento de Ventas e Inventario	2
2.3. Problemática Actual	2
2.4. Sistema Propuesto: Sync-Stock	3
3. Objetivos y Perfiles de Usuario	3
3.1. Objetivos del Sistema	3
3.1.1. Objetivo General	3
3.1.2. Objetivos Específicos	3
3.2. Perfiles de Usuario	3
4. Requerimientos Funcionales	4
5. Modelo de Datos y Mecanismo de Persistencia	6
5.1. Mecanismo de Persistencia y Decisiones de Diseño	6
5.2. Diccionario de Datos (Entidades del Sistema Omnicanal)	6
6. Arquitectura SOA y Mediación a través de un Bus de Servicios (ESB)	7
6.1. El Rol del Enterprise Service Bus (ESB)	7
6.2. Mecanismos Clásicos de Mediación SOA Implementados	9
6.3. Componentes de la Arquitectura	9
6.3.1. Componentes Cliente (Consumidores de Servicios)	9
6.3.2. Componentes de Servicio (Proveedores del Bus)	9
7. Interfaces de los Componentes y Contratos Técnicos	10
7.1. Detalle de Contratos Técnicos de la API	10
7.1.1. 1. Autenticación de Usuarios (Auth Service)	10
7.1.2. 2. Reserva Temporal de Stock (Inventory Service)	12
7.1.3. 3. Confirmación de Reserva (Inventory Service)	12
7.1.4. 4. Consulta de Stock en Tiempo Real (Inventory Service)	13
8. Desarrollo e Implementación de Servicios y Clientes	14
8.1. Infraestructura y Contenerización	14
8.2. Configuración del ESB (RabbitMQ)	14
8.3. Lógica Transaccional y Bloqueos de Concurrencia	14
9. Evidencias de Funcionamiento y Logs del ESB	15
9.1. Flujo 1: Recepción de Venta POS Física (Data Transformer y Message Enricher)	15
9.2. Flujo 2: Expiración de Reserva Web (Mecanismo TTL)	16
10. Análisis Crítico del Sistema y Soluciones	16
11. Guía de Demostración en Vivo	16

12.Conclusiones

17

1. Equipo de Trabajo

Para el desarrollo del sistema Sync-Stock, se ha conformado un equipo de trabajo liderado por el responsable del área de desarrollo:

Integrante	Cargo	Responsabilidades
Fernando Llanca	CEO / Líder de Proyecto	■ Liderazgo estratégico y toma de decisiones ejecutivas. ■ Coordinación directa con los dueños de PescaTotal. ■ Supervisión general del análisis y diseño del sistema.
Nicolas Canelo	Arquitecto de Software	■ Diseño de la estructura técnica y componentes. ■ Selección del stack tecnológico adecuado para tiempo real.
Carlos Jara	Analista de Sistemas	■ Levantamiento y especificación de requerimientos. ■ Definición del modelo de datos y persistencia.
Bastian Peña	Desarrollador Full-stack	■ Implementación de la lógica de negocio y API. ■ Desarrollo de la interfaz de usuario amigable.

2. Descripción del Sistema y Contexto Organizacional

2.1. La Organización: PescaTotal

PescaTotal es una tienda retail especializada en artículos de pesca recreativa y equipamiento outdoor. A pesar de ser una organización de escala pequeña, posee un flujo de ventas constante y una base de clientes fidelizada en la Región Metropolitana.

2.2. Área de Trabajo: Departamento de Ventas e Inventario

Esta unidad operativa gestiona el stock físico de la tienda y, recientemente, la administración del nuevo canal e-commerce. Sus funciones incluyen la recepción de suministros, atención al público y preparación de pedidos online.

2.3. Problemática Actual

La tienda enfrenta dificultades operacionales críticas derivadas de la falta de integración digital:

- **Incumplimiento de pedidos y fricción con clientes por falta de gestión de reservas:** Al carecer de un mecanismo que bloquee o separe físicamente los productos apenas se venden por el canal e-commerce, los artículos quedan expuestos en las vitrinas y son comprados por clientes presenciales. Esto genera un problema grave para el negocio: la necesidad de cancelar compras online ya pagadas, procesar devoluciones de dinero, sobrecargar al personal de atención al cliente y generar un impacto negativo en la reputación y fidelidad de los compradores de PescaTotal.

2.4. Sistema Propuesto: Sync-Stock

Sync-Stock es una plataforma segura de gestión de inventario omnicanal. Su objetivo principal es actuar como un núcleo de datos centralizado que sincronice el stock en tiempo real entre la tienda física y el e-commerce.

Para garantizar la integridad de las operaciones, el sistema contará con acceso restringido mediante roles, entregando a cada empleado (Vendedor, Encargado E-commerce o Administrador) solo las herramientas que necesita. Al registrar una venta en el mesón o recibir un pedido web, el sistema no solo actualiza la disponibilidad global inmediatamente, sino que genera un estado de Reserva inmutable para las compras web, bloqueando la posibilidad de sobreventas en la tienda física. Esto asegurará la coherencia total de los datos y protegerá la experiencia de compra de los clientes.

3. Objetivos y Perfiles de Usuario

3.1. Objetivos del Sistema

3.1.1. Objetivo General

Implementar un sistema integral que garantice la coherencia total del inventario entre la tienda física y el e-commerce de PescaTotal, optimizando la eficiencia operativa y eliminando errores de sobreventa.

3.1.2. Objetivos Específicos

1. Sincronizar de forma automática y bidireccional el stock tras cada transacción.
2. Implementar un módulo de reserva inmediata para asegurar productos vendidos vía web.
3. Proporcionar visibilidad de stock real al personal de ventas mediante una interfaz simple.
4. Reducir en un 85 % las horas-hombre semanales dedicadas a la conciliación y revisión manual de inventarios, alcanzando esta métrica al finalizar el segundo mes de operación en producción del sistema Sync-Stock.”

3.2. Perfiles de Usuario

- **Usuario Primario (Vendedor):** Utiliza el sistema en el mesón de la tienda para consultar disponibilidad y cerrar ventas presenciales.

- Frecuencia de uso: Permanente durante la jornada.
- Nivel tecnico: Intermedio.
- **Usuario Secundario (Encargado E-commerce):** Gestiona los pedidos web, visualiza las reservas y confirma la salida física del producto.
 - Frecuencia de uso: Permanente durante la jornada.
 - Nivel tecnico: Intermedio.
- **Usuario de Apoyo (Administrador):** Gestiona el catálogo, permisos de usuarios y configuraciones de alertas de stock.
 - Frecuencia de uso: Variable, segun necesidades de la tienda.
 - Nivel tecnico: Avanzado.

4. Requerimientos Funcionales

Código	Nombre	Descripción
RF-001	Sincronización Real-Time	El sistema debe descontar el stock del catálogo online inmediatamente tras una venta física confirmada.
RF-002	Reserva de Stock Online	Al detectarse una orden de compra web, el sistema debe mover el producto al estado Reservado”, restándolo del stock físico vendible.
RF-003	Gestión de Productos	Permitir el registro, edición y categorización de artículos con códigos de barra y ubicación física.
RF-004	Control de Despachos	Funcionalidad para confirmar la salida de productos reservados, actualizando el estado a ”Vendido” definitivamente.
RF-005	Alertas de Quiebre	Notificaciones automáticas cuando el stock disponible llegue a un umbral mínimo configurado.
RF-006	Log de Auditoría	Registro detallado de quién, cuándo y por qué canal se realizó cada movimiento de inventario.
RF-007	Autenticación de Usuarios (Login)	El sistema debe requerir que todo usuario inicie sesión mediante credenciales (correo/usuario y contraseña) antes de acceder a cualquier funcionalidad.
RF-008	Control de Acceso Basado en Roles (RBAC)	El sistema debe restringir las funcionalidades e interfaces de acuerdo con el rol del usuario (Administrador, Vendedor, Encargado E-commerce). Por ejemplo, solo el Administrador podrá gestionar permisos o el catálogo.
RF-009	Cierre de Sesión y Tiempos de Inactividad	El sistema debe permitir el cierre de sesión manual y forzar un cierre automático tras 30 minutos de inactividad por motivos de seguridad en los equipos compartidos de la tienda.

5. Modelo de Datos y Mecanismo de Persistencia

Para dar soporte a las funcionalidades del sistema Sync-Stock en un entorno retail omnicanal real, se ha diseñado una estrategia de persistencia transaccional de alta consistencia y un modelo de datos relacional detallado.

5.1. Mecanismo de Persistencia y Decisiones de Diseño

Se ha optado por implementar una **Base de Datos Relacional (RDBMS)** utilizando **PostgreSQL**. Esta decisión responde a la necesidad de resolver problemas complejos de concurrencia y consistencia que un sistema omnicanal enfrenta en tiempo real:

1. **Consistencia Transaccional Estricta (ACID):** Para evitar la sobreventa, los cambios en el inventario deben procesarse con aislamiento transaccional estricto. PostgreSQL permite aislar operaciones mediante el nivel de aislamiento de transacciones *Read Committed* combinado con bloqueos pesimistas explícitos.
2. **Estrategia de Concurrencia y Bloqueos Pesimistas (Pessimistic Locking):** Al procesar una compra en el Punto de Venta (POS) o iniciar un proceso de reserva web, el sistema ejecuta una transacción que bloquea las filas correspondientes de la tabla de inventario mediante la sentencia `SELECT ... FOR UPDATE`.
 - Esto garantiza que ningún otro proceso pueda modificar la cantidad del producto hasta que la transacción actual finalice (haga *COMMIT* o *ROLLBACK*).
 - **Prevención de Deadlocks:** Para evitar bloqueos mutuos (*deadlocks*) bajo cargas de alta concurrencia, el sistema ordena siempre los ítems de la consulta de forma determinista (ordenados por el identificador del producto) antes de solicitar el bloqueo.
3. **Política de Expiración de Reservas y Liberación de Stock (Time-To-Live):** El canal e-commerce genera reservas temporales para proteger el stock de un cliente mientras completa el pago. Sin embargo, para evitar que el stock quede bloqueado indefinidamente si el cliente abandona el carrito, se implementa una regla de expiración de 15 minutos.
 - Cada reserva se registra en la tabla `Reserva` con un campo `FechaExpiracion`.
 - Un proceso daemon (o *cron job*) en el bus de servicios ejecuta periódicamente (cada 1 minuto) una transacción que selecciona las reservas expiradas en estado `Pendiente`, revierte el stock reservado al stock disponible (`StockDisponible = StockDisponible + Cantidad`) y marca la reserva como `Expirada`.
4. **Pooling de Conexiones y Alta Disponibilidad:** Para asegurar tiempos de respuesta menores a 200 ms y soportar picos de carga concurrentes entre la tienda y el sitio web, se utiliza un pooler de conexiones (**PgBouncer**) configurado en modo transacción. Esto permite optimizar el uso de recursos y mantener la latencia en rangos óptimos.

5.2. Diccionario de Datos (Entidades del Sistema Omnicanal)

A continuación se detalla el esquema de base de datos extendido que soporta la lógica del inventario multicanal, reservas y auditoría de transacciones:

Entidad	Atributo	Tipo de Dato	Descripción
Producto	ID_Producto (PK)	UUID	Identificador universal único del artículo.
	SKU	VARCHAR(50)	Código SKU único (Stock Keeping Unit).
	Nombre	VARCHAR(150)	Nombre o descripción del artículo.
	Precio	DECIMAL(12,2)	Precio de venta unitario.
Ubicacion	ID_Ubicacion (PK)	UUID	Identificador único de la ubicación de stock.
	Nombre	VARCHAR(100)	Nombre (ej. 'Bodega Central', 'POS Tienda').
	Tipo	ENUM	Tipo de instalación: 'Tienda' o 'Bodega'.
Inventario	ID_Inventario (PK)	UUID	Identificador del registro de inventario.
	Producto_ID (FK)	UUID	Referencia a la entidad Producto .
	Ubicacion_ID (FK)	UUID	Referencia a la entidad Ubicacion .
	StockDisponible	INT	Unidades físicas disponibles para venta inmediata.
	StockReservado	INT	Unidades retenidas temporalmente por e-commerce.
	StockTransito	INT	Unidades en proceso de traslado entre bodegas.

Cuadro 1: Diccionario de datos: Catálogo, Ubicaciones e Inventario.

6. Arquitectura SOA y Mediación a través de un Bus de Servicios (ESB)

Para resolver la integración y sincronización en tiempo real de múltiples canales, el sistema se ha estructurado bajo el patrón **SOA (Service-Oriented Architecture)** incorporando un **Enterprise Service Bus (ESB)** central como columna vertebral de comunicación. El ESB no solo actúa como canal de mensajería (utilizando el bus de servicios de sockets TCP nativos provisto por la cátedra), sino que implementa componentes de mediación clásicos que desacoplan físicamente y lógicamente a los clientes y servicios.

Principios del ESB en Sync-Stock:

1. **Desacoplamiento de Protocolos:** El bus permite conectar componentes heterogéneos, permitiendo que clientes web interactúen de manera sincrónica (REST/JSON) mientras que la comunicación interna y con servicios del bus se realice mediante sockets TCP nativos.

6.1. El Rol del Enterprise Service Bus (ESB)

En lugar de depender de llamadas HTTP directas y acopladas punto a punto entre la aplicación web e-commerce, el POS físico y los servicios de backend, el ESB centraliza las interacciones y proporciona las siguientes capacidades de infraestructura:

Entidad	Atributo	Tipo de Dato	Descripción
Reserva	ID_Reserva (PK)	UUID	Identificador único de la reserva temporal.
	Producto_ID (FK)	UUID	Referencia a la entidad Producto .
	Ubicacion_ID (FK)	UUID	Referencia a la entidad Ubicacion .
	Cantidad	INT	Cantidad de unidades bloqueadas.
	Estado	ENUM	Estado de la reserva: 'Pendiente', 'Confirmada', 'Expirada'.
	FechaCreacion	TIMESTAMP	Marca de tiempo de la creación de la reserva.
	FechaExpiracion	TIMESTAMP	Límite temporal para consolidar la transacción.
Orden	ID_Orden (PK)	UUID	Identificador único de la orden de venta.
	Canal	ENUM	Origen del pedido: 'POS_Fisico' o 'E-Commerce'.
	Estado	ENUM	Estado del pedido: 'Creada', 'Pagada', 'Despachada', 'Cancelada'.
	FechaCreacion	TIMESTAMP	Marca de tiempo de registro de la orden.
	Total	DECIMAL(12,2)	Monto total de la transacción comercial.
LineaOrden	ID_Linea (PK)	UUID	Identificador de la línea de detalle de la orden.
	Orden_ID (FK)	UUID	Referencia a la entidad Orden .
	Producto_ID (FK)	UUID	Referencia a la entidad Producto .
	Cantidad	INT	Cantidad de unidades compradas.
	PrecioUnitario	DECIMAL(12,2)	Precio facturado para este producto.

Cuadro 2: Diccionario de datos: Reservas temporales y Órdenes de venta.

Entidad	Atributo	Tipo de Dato	Descripción
HistorialInv	ID_Historial (PK)	UUID	Identificador único para auditoría de movimientos.
	Producto_ID (FK)	UUID	Referencia al producto afectado.
	Ubicacion_ID (FK)	UUID	Ubicación física del movimiento.
	Cantidad	INT	Unidades modificadas (valor positivo o negativo).
	TipoMovimiento	ENUM	Tipo de evento: 'Venta', 'Reserva_Creada', 'Reserva_Expirada', 'Recepcion', 'Ajuste'.
	FechaHora	TIMESTAMP	Fecha y hora exacta de la transacción.
	Usuario_ID (FK)	UUID	Identificador del operario responsable.

Cuadro 3: Diccionario de datos: Historial de inventario para log de auditoría.

- **Garantía de Entrega y Asincronía:** Permite que las órdenes del canal digital ingresen a colas de mensajería seguras. Si el servicio de inventario o el sistema externo del courier de despachos sufren una caída temporal, las solicitudes permanecen

encoladas garantizando que ningún pedido se pierda.

- **Desacoplamiento de Protocolos:** El bus permite conectar componentes heterogéneos, permitiendo que clientes web interactúen de manera sincrónica (REST/JSON) mientras que la comunicación interna y con sistemas legados se realice asincrónicamente (AMQP/gRPC).

6.2. Mecanismos Clásicos de Mediación SOA Implementados

El ESB actúa como una capa de mediación activa, aplicando patrones de integración empresarial (EIP) para procesar los mensajes en tránsito:

1. **Enrutamiento Basado en Contenido (Content-Based Router):** Cuando el ESB recibe un mensaje de orden, inspecciona el campo `canal` del payload.
 - Si proviene de `E.Commerce`, enruta el mensaje hacia la cola de validación de reservas y pasarela de pago.
 - Si proviene de `POS_Fisico`, el mensaje es enrutado directamente al canal de facturación local e impresión de boleta física.
2. **Transformación y Traducción de Datos (Data Transformer):** Los canales de venta externos (como Shopify o un POS heredado) envían estructuras de datos propietarias. La capa de mediación del ESB traduce estos formatos a un esquema de datos estándar del sistema (denominado *Modelo Canónico*) antes de que ingresen a los servicios core de inventario.
3. **Enriquecimiento de Mensajes (Message Enricher):** Cuando una orden es capturada por el bus con información básica (ej. SKU y cantidad), el mediador consulta dinámicamente al servicio de catálogo para inyectar datos adicionales (descripción, peso para el courier, precio unitario vigente) antes de despachar el payload completo al servicio de persistencia y despacho.
4. **Puente de Protocolos (Protocol Bridge):** El ESB traduce las peticiones HTTP REST síncronas provenientes del frontend POS en mensajes TCP estructurados dirigidos a los servicios del bus, y viceversa, permitiendo que el sistema se comunique eficientemente sin acoplar tecnológicamente sus extremos.

6.3. Componentes de la Arquitectura

Bajo este esquema de mediación, los componentes del ecosistema SOA se dividen en:

6.3.1. Componentes Cliente (Consumidores de Servicios)

- **Cliente Web POS (Punto de Venta):** Interfaz local en la tienda física para vendedores. Consume servicios a través del ESB de forma síncrona para consultas de stock rápidas y de forma asíncrona para el cierre de caja.
- **Panel de Administración (Dashboard):** Aplicación web para el encargado de e-commerce y administradores. Permite monitorear el flujo de colas en el ESB, verificar alertas de stock y gestionar reservas en tiempo real.

6.3.2. Componentes de Servicio (Proveedores del Bus)

- **Auth Service:** Servicio transversal expuesto en el bus para verificar credenciales y gestionar roles de acceso (RBAC).
- **Inventory Service:** Servicio core que gestiona el inventario omnicanal físico y web, respondiendo a los comandos de reserva y confirmación enviados por el ESB.
- **Sales & Order Service:** Orquesta el ciclo de venta, consumiendo el Inventory Service mediante el ESB. Si una venta web falla en la pasarela de pago, este servicio coordina la compensación (cancelación de reserva).

7. Interfaces de los Componentes y Contratos Técnicos

Para garantizar la interoperabilidad del sistema Sync-Stock en un entorno omnicanal crítico en tiempo real, se definen contratos técnicos rigurosos. A continuación, se resume la relación entre interfaces, Requerimientos Funcionales (RF) y Requerimientos No Funcionales (RNF):

Servicio	Endpoint / Operación	RF Satisfecho	RNF Mapeado
Auth Service	POST /api/v1/auth/login	RF-007, RF-008	Seguridad, Expiración (30 min)
Inventory Service	POST /api/v1/inventory/reserve	RF-002	Consistencia Estricta, Alta Concurrencia
Inventory Service	POST /api/v1/inventory/commit	RF-004, RF-006	Integridad Transaccional, Auditoría
Inventory Service	GET /api/v1/inventory/stock/{sku}	RF-001, RF-003	Baja Latencia (< 200 ms)

Cuadro 4: Resumen de endpoints de servicios, RFs y RNFs.

7.1. Detalle de Contratos Técnicos de la API

A continuación se especifican las estructuras de las peticiones, las respuestas de éxito y el control de errores detallado (en formato JSON) para cada interfaz.

7.1.1. 1. Autenticación de Usuarios (Auth Service)

Endpoint: POST /api/v1/auth/login

Descripción: Valida las credenciales del operario y devuelve un token firmado (JWT) con su rol asociado (RBAC).

Estructura de la Petición (Request):

```
{
  "username": "fllancao",
```

```
"password": "mi_clave_segura_123"
}
```

Respuesta de Éxito (200 OK):

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJhMWIy...[truncado]",
  "usuario": {
    "id": "e4b2d288-43ea-46b2-a5e2-88229988ff22",
    "nombre": "Fernando Llancao",
    "rol": "Administrador"
  },
  "expiraEnSegundos": 1800
}
```

Respuesta de Error (401 Unauthorized):

```
{
  "codigoError": "CREDENCIALES_INVALIDAS",
  "mensaje": "El usuario o la contraseña ingresados son incorrectos.",
  "timestamp": "2026-06-03T01:10:00Z"
}
```

7.1.2. 2. Reserva Temporal de Stock (Inventory Service)

Endpoint: POST /api/v1/inventory/reserve

Descripción: Reserva temporalmente unidades de un producto en una ubicación específica para evitar la sobreventa desde el canal web.

Estructura de la Petición (Request):

```
{
  "ordenId": "d3b07384-d113-49c5-a5d6-d68aee460183",
  "items": [
    {
      "sku": "PES-CANA-CARBON-01",
      "cantidad": 2,
      "ubicacionId": "e2a22288-e22a-43c2-bf72-888e422201b1"
    }
  ]
}
```

Respuesta de Éxito (201 Created):

```
{
  "reservaId": "b1b88220-432a-4632-a5e2-88229988ff22",
  "estado": "Pendiente",
  "fechaCreacion": "2026-06-03T01:10:00Z",
  "fechaExpiracion": "2026-06-03T01:25:00Z"
}
```

Respuesta de Error - Stock Insuficiente (409 Conflict):

```
{
  "codigoError": "STOCK_INSUFICIENTE",
  "mensaje": "No hay suficiente stock para el producto indicado en esa bodega.",
  "timestamp": "2026-06-03T01:10:01Z",
  "detalles": {
    "sku": "PES-CANA-CARBON-01",
    "stockDisponible": 1,
    "cantidadSolicitada": 2
  }
}
```

7.1.3. 3. Confirmación de Reserva (Inventory Service)

Endpoint: POST /api/v1/inventory/commit

Descripción: Transforma una reserva temporal en una venta confirmada y descontada definitivamente.

Estructura de la Petición (Request):

```
{
  "reservaId": "b1b88220-432a-4632-a5e2-88229988ff22",
  "ordenId": "d3b07384-d113-49c5-a5d6-d68aee460183"
}
```

Respuesta de Éxito (200 OK):

```
{
  "status": "success",
  "mensaje": "Reserva confirmada e inventario rebajado definitivamente.",
  "transaccionId": "c8e228c2-482a-466d-88b1-2299aaffd883"
}
```

Respuesta de Error - Reserva Expirada o Inexistente (410 Gone):

```
{
  "codigoError": "RESERVA_NO_VALIDA",
  "mensaje": "La reserva no existe, ya fue consolidada o expiró tras el límite de 15 m",
  "timestamp": "2026-06-03T01:10:05Z",
  "detalles": {
    "reservaId": "b1b88220-432a-4632-a5e2-88229988ff22",
    "estadoActual": "Expirada"
  }
}
```

7.1.4. 4. Consulta de Stock en Tiempo Real (Inventory Service)

Endpoint: GET /api/v1/inventory/stock/{sku}

Descripción: Devuelve la disponibilidad detallada de un SKU en todas las ubicaciones físicas y bodegas registradas.

Respuesta de Éxito (200 OK):

```
{
  "sku": "PES-CANA-CARBON-01",
  "nombre": "Caña de Pescar Carbono Pro",
  "precio": 45000.00,
  "inventario": [
    {
      "ubicacionId": "e2a22288-e22a-43c2-bf72-888e422201b1",
      "ubicacionNombre": "Tienda Santiago POS",
      "stockDisponible": 15,
      "stockReservado": 2,
      "stockTransito": 0
    },
    {
      "ubicacionId": "a5d3f288-43ea-46b2-a5e2-88229988abcd",
      "ubicacionNombre": "Bodega Principal E-commerce",

```

```
    "stockDisponible": 50,  
    "stockReservado": 8,  
    "stockTransito": 10  
  }  
]  
}
```

8. Desarrollo e Implementación de Servicios y Clientes

El sistema **Sync-Stock** ha sido completamente implementado y desplegado de forma contenerizada para garantizar la consistencia en el desarrollo y producción. A continuación, se detalla la configuración del entorno, la arquitectura de mensajería del ESB y la lógica transaccional de persistencia.

8.1. Infraestructura y Contenerización

Para el despliegue del sistema, se diseñó una plantilla de orquestación multibastión mediante Docker Compose, definiendo los siguientes componentes:

- **Motor PostgreSQL 15-alpine:** Expuesto en el puerto 5432, encargado del almacenamiento relacional consistente.
- **ESB TCP (soabus:v1):** Contenedor del bus de servicios expuesto en el puerto 5000, implementando el protocolo de sockets TCP nativo para coordinar la mensajería entre los clientes y los servicios.
- **Backend en Node.js y TypeScript:** Servidor Express y cliente/servidor ESB TCP ejecutándose en el puerto 5001.
- **Frontend en React y Vite:** Interfaz cliente ejecutándose en el puerto 3000.

8.2. Configuración del ESB (Protocolo de Sockets TCP)

La comunicación con el bus de servicios se implementa utilizando un protocolo propietario sobre sockets TCP nativos en el puerto 5000. El bus espera mensajes con la siguiente estructura rígida de caracteres:

nnnnnSSSSDATOS

Donde **nnnnn** es un prefijo de 5 dígitos alineado con ceros indicando el largo de la carga útil, **SSSSS** representa el nombre del servicio de destino (de 5 caracteres fijos) y **DATOS** contiene el payload (generalmente un objeto JSON serializado).

Para operar en la arquitectura, se han registrado tres microservicios principales usando el comando de inicialización **sinit**:

1. **autho:** Servicio de autenticación y verificación de credenciales JWT.
2. **inven:** Servicio de control de inventario omnicanal, reservas y confirmaciones.
3. **sales:** Servicio de registro de ventas, órdenes y logs de auditoría.

8.3. Lógica Transaccional y Bloqueos de Concurrency

Para evitar la sobreventa, el servicio de inventario implementa la lógica de bloqueo de fila pesimista en PostgreSQL utilizando la sentencia:

```
SELECT * FROM "Inventario"  
WHERE "producto_id" = $1 AND "ubicacion_id" = $2  
FOR UPDATE;
```

Esta consulta bloquea la fila hasta que la transacción se completa (ya sea con un commit o un rollback).

Para evitar condiciones de *deadlock* cuando se registran múltiples productos en una sola orden del POS físico, el backend ordena determinísticamente todos los artículos basándose en su identificador UUID antes de solicitar los bloqueos. Esto garantiza que las transacciones concurrentes siempre adquieran los recursos en el mismo orden lineal, eliminando ciclos de espera mutua.

9. Evidencias de Funcionamiento y Logs del ESB

La mediación SOA sobre sockets TCP nativos fue validada mediante pruebas de integración locales. A continuación, se presentan las trazas reales del comportamiento del bus durante el procesamiento de ventas y el flujo del mediador.

9.1. Flujo 1: Registro de Servicios en el Bus TCP (Inicialización)

Al levantar el backend de Sync-Stock, los tres módulos funcionales se conectan al bus en el puerto 5000 y se registran dinámicamente:

```
[SOABus Service autho] Connecting to ESB on port 5000...  
[SOABus Service autho] Connected. Registering service...  
[SOABus Service autho] Registered successfully. ESB Confirmation: sinit0Kautho  
  
[SOABus Service inven] Connecting to ESB on port 5000...  
[SOABus Service inven] Connected. Registering service...  
[SOABus Service inven] Registered successfully. ESB Confirmation: sinit0Kinven  
  
[SOABus Service sales] Connecting to ESB on port 5000...  
[SOABus Service sales] Connected. Registering service...  
[SOABus Service sales] Registered successfully. ESB Confirmation: sinit0Ksales
```

9.2. Flujo 2: Recepción de Venta POS Física (Data Transformer y Message Enricher)

Cuando se ingresa una venta en el POS, la API REST en el puerto 5001 actúa como puente abriendo un socket temporal hacia el ESB. Envía la transacción estructurada al servicio `sales`. El mediador decodifica el frame TCP y procesa la venta:

```
[ESB Content-Based Router] Received local order TCP frame:  
00127sales{"action":"checkout","items":[{"sku":"PES-CA1","cantidad":2}],
```

```
"ubicacion": "Tienda Valdivia"}
```

```
[ESB Message Enricher] Enriched payload: {
```

```
  "items": [  
    {  
      "sku": "PES-CA1",  
      "cantidad": 2,  
      "precio": 59990,  
      "nombre": "Caña de Pescar Shakespeare Ugly Stik"  
    }  
  ],  
  "location": "Tienda Valdivia",  
  "timestamp": "2026-06-03T05:15:30.012Z"
```

```
}
```

```
[ESB Mediator] Order processed successfully: ID 9e4f2010-8b1b-4d43-9cd3-488f55ba9902
```

9.3. Flujo 3: Expiración de Reserva Web (Mecanismo TTL)

El demonio de expiración de reservas escanea la base de datos de manera periódica. Al detectar que una reserva en bodega ha superado su límite temporal de 15 minutos sin ser pagada, libera automáticamente el stock disponible. La consola de base de datos y logs evidencia el rollback:

```
[Expiration Daemon] Reservation b1b88220-432a-4632-a5e2-88229988ff22 expired.  
Reverting reserved stock...  
[Expiration Daemon] Reservation b1b88220-432a-4632-a5e2-88229988ff22 stock  
reverted successfully.
```

En el log de auditoría se inserta un registro con tipo de movimiento `Reserva_Expirada`, permitiendo la trazabilidad del inventario.

10. Análisis Crítico del Sistema y Soluciones

Durante el desarrollo del proyecto, surgieron desafíos técnicos complejos debido al comportamiento en tiempo real de la omnicanalidad y el uso de sockets nativos:

1. **Rendimiento vs Consistencia:** El uso de bloqueos pesimistas (`FOR UPDATE`) garantiza que no habrá sobreventa. Sin embargo, introduce un cuello de botella si se generan miles de solicitudes concurrentes sobre el mismo producto. Se mitigó configurando un Connection Pool en PostgreSQL y limitando el tiempo de espera de los bloqueos (`lock_timeout`).
1. **Control de Flujo TCP y Buffer Fragmentation:** Los sockets TCP transmiten datos como un stream continuo de bytes. Si la red se congestiona o los payloads JSON son muy grandes, los mensajes se fragmentan. Se solucionó implementando un analizador de buffer que acumula los datos entrantes y solo extrae e interpreta la carga útil cuando el largo acumulado coincide con los 5 caracteres iniciales del protocolo `nnnnn`. Cabe destacar que

se detectó una limitación en el binario precompilado del ESB, el cual no acumula de manera correcta los buffers de respuestas de salida superiores a ~1010 bytes, tratando los fragmentos subsecuentes como mensajes independientes de trama errónea. Esto se solucionó optimizando las consultas SQL/Prisma en el Mediador del Backend para seleccionar solo los campos críticos requeridos por el frontend (SKU, nombre, precio e inventarios simplificados), reduciendo el payload de productos de 2286 bytes a 659 bytes, lo que asegura su transmisión completa dentro de un único paquete de red y elimina fallos de comunicación.

11. Guía de Demostración en Vivo

Para validar el funcionamiento en vivo del sistema en la presentación final frente a la comisión evaluadora, se diseñó el siguiente guion de pruebas:

Paso 1: Inicialización del Entorno Ejecutar los contenedores y verificar que los servicios estén listos para aceptar peticiones:

```
docker-compose up -d
```

Paso 2: Simulación de Concurrencia y Bloqueo Pesimista

1. Acceder a la interfaz del Dashboard Administrador en el puerto 3000.
2. Utilizar el panel de simulación de e-commerce para reservar 5 unidades del producto *Caña de Pescar Shakespeare Ugly Stik* (SKU: PES-CA1) en la "Tienda Valdivia" (Stock disponible inicial: 10).
3. Verificar en tiempo real que el stock disponible baja instantáneamente a 5 y el stock reservado sube a 5.
4. En paralelo, intentar realizar una venta desde el panel del POS físico por 7 unidades del mismo producto en la misma ubicación. El sistema lanzará de inmediato una alerta visual: *STOCK_INSUFICIENTE, impidiendo transacciones inválidas*.

Paso 3: Consolidación de Compra y Liberación

1. En la pestaña de reservas del administrador, seleccionar la reserva pendiente y hacer clic en **Consolidar Pago**.
2. El mediador procesará asincrónicamente el mensaje, creará la orden de venta definitiva, reducirá el stock reservado a 0 y guardará el registro en el log de auditoría.

12. Conclusiones

Con la definición de este modelo relacional omnicanal, el desarrollo del backend con lógica transaccional anti-deadlock, y la mediación asíncrona implementada mediante sockets TCP nativos con el ESB, el proyecto **Sync-Stock** cumple a cabalidad con los rigurosos estándares de la arquitectura orientada a servicios. La plataforma no solo resuelve la problemática de sobreventa del caso comercial PescaTotal, sino que establece bases escalables para la incorporación ágil de nuevos canales de venta y automatización logística en el futuro.